

Esame Programmazione di Sistema / API Programming

Data: 2022-06-20

QUESTIONS

Domanda 1

Si definisca il concetto di Smart Pointer, quindi si fornisca un esempio (Rust o C++) che ne evidenzi il ciclo di vita.

Domanda 2

Si illustrino le differenze nel linguaggio Rust tra `std::sync::mpsc::channel()` e `std::sync::mpsc::sync_channel()` indicando quali tipi di sincronizzazione i due meccanismi permettono.

Domanda 3

Dato il seguente frammento di codice Rust (ogni linea è preceduta dal suo indice):

```
1. struct Point {
2.     x: i16,
3.     y: i16,
4. }
5.
6. enum PathCommand {
7.     Move(Point),
8.     Line(Point),
9.     Close,
10. }
11.
12. let mut v = Vec::<PathCommand>::new();
13. v.push(PathCommand::Move(Point{x: 1, y: 1}));
14. v.push(PathCommand::Line(Point{x: 10, y: 20}));
15. v.push(PathCommand::Close);
16. let slice = &v[..];
```

Si descriva il contenuto dello stack e dello heap al termine dell'esecuzione della riga 16.

Domanda 4

Un paradigma frequentemente usato nei sistemi reattivi è costituito dall'astrazione detta **Looper**.

Quando viene creato, un **Looper** crea una coda di oggetti generici di tipo **Message** ed un thread. Il thread attende - senza consumare cicli di CPU - che siano presenti messaggi nella coda, li estrae a uno a uno nell'ordine di arrivo, e li elabora. Il costruttore di **Looper** riceve due parametri, entrambi di tipo (puntatore a) funzione: `process(...)` e `cleanup()`. La prima è una funzione responsabile di elaborare i singoli messaggi ricevuti attraverso la coda; tale funzione accetta un unico parametro in ingresso di tipo **Message** e non ritorna nulla. La seconda è una funzione priva di argomenti e valore di ritorno e verrà invocata dal thread incapsulato nel **Looper** quando esso starà per terminare.

Looper offre un unico metodo pubblico, thread safe, oltre a quelli di servizio, necessari per gestirne il ciclo di vita: `send(msg)`, che accetta come parametro un oggetto generico di tipo **Message** che verrà inserito nella coda e successivamente estratto dal thread ed inoltrato alla funzione di elaborazione.

Quando un oggetto **Looper** viene distrutto, occorre fare in modo che il thread contenuto al suo interno invochi la seconda funzione passata nel costruttore e poi termini.

Si implementi, utilizzando il linguaggio Rust o C++, tale astrazione tenendo conto che i suoi metodi dovranno essere *thread-safe*.

SOLUTIONS

Soluzione 1

Uno Smart Pointer è un tipo di dato astratto che simula il comportamento di un puntatore, fornendo al contempo funzionalità aggiuntive, come la gestione automatica e sicura della memoria o il reference counting. In Rust, gli Smart Pointer sono strutture dati che implementano tipicamente i trait `Deref` e `Drop`. `Deref` permette allo smart pointer di comportarsi come un riferimento ordinario (`&T`), mentre `Drop` consente di eseguire codice di pulizia (come la deallocazione della memoria nello heap) quando lo smart pointer esce dal suo ambito di validità (scope). Gli esempi più comuni in Rust sono `Box<T>`, `Rc<T>`, `Arc<T>`.

Esempio in Rust con `Box<T>` che ne evidenzia il ciclo di vita:

```
1 struct MyStruct {
2     value: i32,
3 }
4
5 impl Drop for MyStruct {
6     fn drop(&mut self) {
7         println!("MyStruct con valore {} distrutta!", self.value);
8     }
9 }
10
11 fn main() {
12     {
13         // Allocazione: viene allocato uno smart pointer Box nello stack,
14         // mentre i dati effettivi di MyStruct vengono allocati nello heap.
15         let my_box = Box::new(MyStruct { value: 42 });
16
17         // Uso: grazie al trait Deref, Box può essere usato
18         // in modo trasparente per accedere ai dati puntati.
19         println!("Valore: {}", my_box.value);
20
21         // Fine scope: qui my_box esce dallo scope.
22         // Il trait Drop viene chiamato automaticamente su my_box,
23         // che a sua volta chiama il Drop di MyStruct:
24         // 1. Viene stampato "MyStruct con valore 42 distrutta!"
25         // 2. La memoria nello heap allocata dal Box viene deallocata in
26         modo sicuro.
27     }
28     println!("Fine del main.");
29 }
```

Soluzione 2

In Rust, il modulo `std::sync::mpsc` (*multi-producer, single-consumer*) fornisce due tipi principali di canali per la comunicazione tra thread. Essi si differenziano fondamentalmente per la capacità del buffer e per il tipo di sincronizzazione che offrono:

- **channel()** (**Canale Asincrono / Unbounded**): Crea un canale con capacità virtualmente infinita (limitata solo dalla memoria disponibile di sistema). Il metodo `send()` sul `Sender` è **non bloccante**. Questo significa che offre una **sincronizzazione asincrona**: il thread mittente può inserire un messaggio nel canale e proseguire immediatamente la propria esecuzione senza mai doversi fermare ad aspettare che il consumatore legga il messaggio.
- **sync_channel(bound)** (**Canale Sincrono / Bounded**): Crea un canale con capacità limitata, pari al valore `bound` fornito. Il metodo `send()` sul `SyncSender` blocca il thread chiamante qualora il buffer interno sia attualmente pieno, mettendolo in attesa finché il ricevitore non estrae un messaggio liberando così spazio nel canale. Se `bound` è pari a 0 (noto come *Rendezvous channel*),

il mittente si blocca *sempre* finché il ricevitore non si appresta ad estrarre il messaggio nello stesso istante, offrendo così una **sincronizzazione stretta (rendezvous)**. Il canale bounded risulta molto utile per implementare policy di *backpressure*, ovvero per rallentare i produttori se i consumatori non riescono a tenere il passo, evitando così consumi incontrollati di memoria.

Soluzione 3

Per descrivere il layout in memoria è prima necessario analizzare le dimensioni dei tipi:

- **Point** è costituito da due `i16` (2 byte ciascuno), quindi ha un allineamento di 2 byte e una dimensione totale di 4 byte.
- **PathCommand** è un `enum`. In Rust, gli `enum` richiedono uno spazio (tag discriminante) per distinguere la variante corrente, più lo spazio sufficiente ad ospitare il payload della variante più grande. Il tag occupa 1 byte (più eventuale padding per mantenere l'allineamento). Le varianti `Move(Point)` e `Line(Point)` necessitano del massimo payload di 4 byte. La dimensione totale dell'enum ammonta a 6 byte (2 byte per il tag/padding + 4 byte di payload). L'allineamento di `PathCommand` corrisponde a 2 byte.

Analisi dello stato della memoria al termine dell'esecuzione della **riga 16** (`let slice = &v[..];`):

Nello Stack: Si trovano le due variabili locali della funzione corrente:

- **v (Vec<PathCommand>):** Un `Vec` in Rust è composto internamente da tre campi di tipo `usize` (8 byte ciascuno su architetture a 64-bit, totale 24 byte):
 1. Un **puntatore** che punta all'inizio del buffer contiguo allocato nello *heap*.
 2. **Capacity (capacità):** Indica quanti elementi al massimo possono essere contenuti nel buffer prima di richiedere una nuova ri-allocazione (presumibilmente ≥ 3 , per esempio 4).
 3. **Len (lunghezza):** Pari a 3, dato che sono stati inseriti tre elementi.
- **slice (&[PathCommand]):** E' un "*fat pointer*" (puntatore grasso) relativo ad una vista (*slice*) sui dati del vettore. È composto da due campi di tipo `usize` (totale 16 byte su 64-bit):
 1. Un **puntatore** che punta agli stessi dati presenti nello heap puntati da `v`.
 2. **Len (lunghezza):** Pari a 3, che indica la porzione di array considerata.

Nello Heap: Nello *heap* si trova l'allocazione dinamica gestita in via esclusiva dal `Vec`. È un array contiguo di memoria la cui dimensione in byte è data da `capacity * size_of::<PathCommand>()`. Supponendo una capacità di 4 elementi, avremo allocati $4 \times 6 = 24$ byte.

I primi 3 slot dell'array ($3 \times 6 = 18$ byte) sono inizializzati e contengono in sequenza i tre elementi inseriti:

1. Al primo offset, la variante `Move`. Nel layout in memoria ci sarà il tag (es. valore 0) e poi i byte corrispondenti a `Point{x: 1, y: 1}`.
2. Di seguito, la variante `Line`. Vi si troverà il tag corrispondente (es. valore 1) e i byte del `Point{x: 10, y: 20}`.
3. Infine, la variante `Close`. Il suo tag (es. valore 2) prenderà posto indicando tale variante. Il resto dello spazio associato al payload di questo elemento è non utilizzato / contiene dati non definiti a causa dell'assenza di dati per `Close`.

Soluzione 4

L'astrazione del **Looper** implementata in Rust richiede di gestire in modo coordinato la comunicazione sicura tra thread, l'incapsulamento del ciclo vita del thread in background e la gestione pulita della terminazione al rilascio delle risorse. Questo obiettivo è raggiungibile adoperando una coda mpsc (`std::sync::mpsc::channel`) e la gestione dei lifetime e ownership del linguaggio.

```
1 use std::sync::mpsc::{channel, Sender};
2 use std::thread::{self, JoinHandle};
3
4 pub struct Looper<Message: Send + 'static> {
5     // Opzionali per poterne estrarre il valore (tramite take)
6     // nel costruttore Drop, gestendo così in sicurezza il possesso.
7     sender: Option<Sender<Message>>,
8     handle: Option<JoinHandle<()>>,
9 }
10
11 impl<Message: Send + 'static> Looper<Message> {
12     pub fn new(
13         process: fn(Message),
14         cleanup: fn(),
15     ) -> Self {
16         // channel() crea un canale asincrono asimmetrico per la
17         // comunicazione di messaggi
18         let (sender, receiver) = channel();
19
20         // Spawn del thread in background che agirà da ricevitore
21         let handle = thread::spawn(move || {
22             // recv() si blocca, senza sprecare cicli di CPU, in attesa che
23             // vengano inseriti messaggi nel canale da send(),
24             // Estrae uno alla volta ed elaborandoli attraverso "
25             process"
26             while let Ok(msg) = receiver.recv() {
27                 process(msg);
28             }
29
30             // Quando il sender di canale viene deallocato/rilasciato, recv
31             // restituisce un Err interrompendo il ciclo.
32             // Possiamo così invocare la funzione di terminazione e uscire
33             // dal thread.
34             cleanup();
35         });
36
37         Self {
38             sender: Some(sender),
39             handle: Some(handle),
40         }
41
42     pub fn send(&self, msg: Message) {
43         // La funzione send richiede solo &self (riferimento immutabile)
44         // in quanto il tipo Sender è intrinsecamente thread-safe (cloneable).
45         if let Some(s) = &self.sender {
46             // Ignoriamo i potenziali SendError, che si verificano qualora
47             // il thread ricevitore terminasse preventivamente la sua
48             // esecuzione in modo anomalo
49             let _ = s.send(msg);
50         }
51     }
52 }
```

```

48     }
49 }
50
51 // L'implementazione del trait Drop gestisce il ciclo di vita
52 // e la chiusura coordinata delle risorse e la terminazione del thread
53 // contenuto.
54 impl<Message: Send + 'static> Drop for Looper<Message> {
55     fn drop(&mut self) {
56         // Chiamando take(), otteniamo il possesso del Sender sottraendolo
57         // alla struttura. Una volta uscito dallo scope di questo blocco, s verrà
58         // distrutto chiudendo il canale.
59         if let Some(s) = self.sender.take() {
60             drop(s);
61         }
62
63         // Dopo la deallocazione del canale, il thread consumer si sbloccherà
64         // dalla recv() con un Err(RecvError), invocherà cleanup e terminerà
65         // l'ordine.
66         // E' quindi lecito chiamare join() per attenderne un'uscita in
67         // ordine.
68         if let Some(handle) = self.handle.take() {
69             let _ = handle.join();
70         }
71     }
72 }

```